JissrON

Learning Management System

Technical Delivery Documentation

Version 1.0 · May 2026

1. Overview

JissrON is a full-stack Learning Management System hosted at **jissron.com**. It supports three independent revenue streams: on-demand video courses, scheduled live sessions (Zoom/Meet), and 1-on-1 consultations. The platform is bilingual (Arabic, French, English) and bi-currency (MAD, USD), with payment integrations for both the Moroccan market (CMI) and international customers (Stripe).

This document describes the implementation: the technology stack, architecture decisions, data model, security posture, deployment topology, and operational requirements.

2. Technology Stack

2.1 Application framework

- **Next.js 15** (App Router) — React 19 server components, server actions, dynamic and static rendering per route
- **TypeScript 5** — strict mode, no `any` in domain code
- **Tailwind CSS 3** — utility-first styling, custom design tokens (`primary: #003d80` , `ink: #081a36`)
- **shadcn/ui** — accessible primitives (Dialog, Drawer, Form, etc.), copied into the repo, not a dependency
- **react-hook-form + zod** — typed form validation, used in every admin form

2.2 Data layer

- **PostgreSQL 15** hosted on Supabase (EU-West-1)
- **Prisma ORM 6** — type-safe queries, migrations via `prisma db push`
- Connection pooling via Supabase's pgBouncer (`DATABASE_URL`), direct connection via `DIRECT_URL` for migrations

2.3 Authentication

- **NextAuth.js v5** with `PrismaAdapter`
- Email magic-link (Resend transport), Google OAuth, LinkedIn OAuth
- Role-based access: `STUDENT` / `INSTRUCTOR` / `ADMIN`
- Sessions stored in DB (`Session` table), JWT for the cookie

2.4 Payments

- **Stripe** (international USD) — hosted Checkout Sessions, Webhook-driven fulfillment
- **CMI** (Moroccan MAD cards) — hosted payment page with SHA-512 hash signing, server-side store key
- **Bank transfer** (MAD) — admin-confirmed via `/admin/orders/[id]`

2.5 Media & uploads

- **UploadThing** — direct-to-cloud uploads for avatars, course thumbnails, lesson audio, PDFs, assignment submissions
- **Bunny.net Stream** — video hosting for course lessons (signed embed URLs, never exposes API key)

2.6 Email

- **Resend** — transactional email (orders, payments, completions, reminders)
- Verified sender configured in `EMAIL_FROM`

2.7 Hosting

- **Vercel** (Hobby plan) — production deployment, edge CDN, serverless functions
- **Supabase** — Postgres + connection pooling
- **cron-job.org** — external scheduled trigger for `/api/cron/live-session-reminders` (workaround for Hobby plan cron limits)

3. Repository Structure

```
/
├── app/
│   ├── (marketing)/        Public pages: /, /courses, /live, /consultants,
│   │                       /contact, /p/[slug], /dashboard, /checkout
│   ├── (admin)/admin/      Admin panel: settings, courses, users, orders,
│   │                       live, consultants, grading, payouts, analytics
│   ├── (auth)/             /signin, /signup, /verify-request, /welcome
│   ├── (learn)/courses/    /courses/[slug]/learn — full lesson viewer
│   ├── instructor/         Instructor read-only dashboard
│   ├── certificates/[serial]/  Public certificate verification
│   └── api/
│       ├── auth/[...nextauth]/
│       ├── webhooks/stripe/        Stripe webhook handler
│       ├── cmi/callback/           CMI payment callback
│       ├── cron/live-session-reminders/
│       └── uploadthing/
├── components/
│   ├── ui/                 shadcn primitives
│   ├── marketing/          Public site components
│   ├── dashboard/          Student dashboard
│   ├── admin/              Admin forms, tables, dialogs
│   ├── learn/              Lesson viewer (Video, Audio, PDF, Quiz, Assignment, etc.
│   ├── live/               Live session components
│   └── consultants/        Consultant booking widget
├── lib/
│   ├── actions/            Server actions (orders, bookings, consults, reviews,
│   │                       refunds, certificates, fulfill-order, contact, etc.)
│   ├── data/               Read-only data loaders (cached server queries)
│   ├── emails/senders.ts   Resend email templates
│   ├── auth.ts             NextAuth config
│   ├── auth/access.ts      Enrollment gating
│   ├── db.ts               Prisma client singleton
│   ├── cmi.ts              CMI hash sign + verify
│   ├── stripe.ts           Stripe client factory
│   ├── bunny.ts            Bunny embed URL generator
│   └── currency.ts         MAD/USD formatting
├── prisma/
│   └── schema.prisma       Database schema (40+ models)
├── scripts/
│   └── seed-legal-pages.ts Seeder for /p/privacy and /p/terms
└── docs/                   Internal specs and delivery docs
```

4. Data Model

The Prisma schema defines ~40 models grouped into the following domains. All IDs use cuid (collision-resistant random strings).

4.1 Identity

Model	Purpose
`User`	Account record; one per email. `role` enum: STUDENT/INSTRUCTOR/ADMIN. `platformCutPercent` for instructor revenue split.
`Account`	NextAuth OAuth credentials per provider (Google, LinkedIn).
`Session`	NextAuth session tokens.
`ActivityLog`	Audit log for admin-side mutations.

4.2 Content

Model	Purpose
`Course`	Top-level container. Owned by an instructor. Pricing in MAD and USD cents. Optional `stripePriceId`.
`Module`	Ordered groups of lessons within a course.
`Lesson`	Individual learning unit. `type` enum: VIDEO, AUDIO, PDF, HTML, TEXT, QUIZ, ASSIGNMENT.
`Quiz` / `QuizQuestion` / `QuizAttempt`	Quiz definitions, questions (single/multiple/text), and student attempts with auto-grading.
`Assignment` / `AssignmentSubmission`	File-upload assignments with manual grading.
`Category`	Course taxonomy for browse filters.
`CourseFAQ`	Per-course FAQ entries.
`LessonQuestion` / `LessonQuestionReply`	Per-lesson Q&A threads.

4.3 Commerce

Model	Purpose
`Order`	One unified model for all paid transactions. Carries `courseId`, `liveSessionId`, OR `consultBookingId`. `paymentMethod` enum: NONE / BANK_TRANSFER / CMI / STRIPE. CMI fields: `cmiTransactionId`, `cmiResponseRaw`. `instructorPayoutAt` tracks revenue split status.
`Enrollment`	Active enrollment in a course. `method` mirrors how it was obtained. Unique on `(userId, courseId)`.
`LessonProgress`	Per-lesson watched seconds + completed flag.
`Certificate`	Issued on course completion. Snapshots student/course/instructor names + serial.
`Counter`	Atomic counter for human-readable order references (`JIS-2026-XXXX`).

4.4 Live & 1-on-1

Model	Purpose
`LiveSession`	Scheduled session. `kind`: AMA/WORKSHOP/SEMINAR/COHORT. `seatsTotal` enforces capacity. `meetingUrl` revealed only to bookees in the join window.
`Booking`	One per (user, liveSession). `reminderSentAt` for idempotent reminder cron.
`Consultant`	Profile linked to a User. `availability` JSON of `{day, slots: [{start, end}]}`. `ratePerSessionMadCents` / `ratePerSessionUsdCents`.
`ConsultBooking`	30-minute 1-on-1 slot. `status`: PENDING (awaiting payment) / CONFIRMED.

4.5 CMS & site config

Model	Purpose
`SiteSettings`	Singleton (`id: "default"`). Holds every editable string on the public site: brand, hero copy, mid-CTA, footer columns, SEO, payment gateway

	credentials, support contact.
`Page`	CMS-driven static pages rendered at `/p/[slug]`. Currently houses Privacy and Terms.
`FAQ`	Global FAQ entries.
`Review`	Course reviews. Unique on `(userId, courseId)`. Gated to enrolled-and-completed students.

5. Routing

5.1 Public routes

Path	Purpose
`/`	Homepage. Hero, featured courses, upcoming live, featured consultants, mid-CTA, FAQ.
`/courses`	Course catalog with filters (category, level, price, language, payment method, rating, duration), pagination, full-text search via `?search=`.
`/courses/[slug]`	Course detail. What you'll learn, instructor card, modules tree, reviews (read + write), FAQ, sticky enroll widget.
`/courses/[slug]/learn`	Lesson viewer (enrolled only). Auto-resumes, completion tracking, lesson Q&A, suggested courses.
`/live`	Live session catalog. Upcoming + recent recordings.
`/live/[slug]`	Live session detail with seat counter, booking, paid checkout, meeting link reveal.
`/consultants`	Consultant directory. Featured first, with skill tags and availability hints.
`/consultants/[id]`	Consultant profile with bookable 30-min slot picker over next 14 days.
`/contact`	Contact form posting to support email; honeypot anti-spam.
`/p/[slug]`	CMS-rendered pages. Currently `/p/privacy` and `/p/terms`. Shortcuts: `/privacy`, `/terms`.
`/certificates/[serial]`	Public verification page for issued certificates. Print-ready.
`/dashboard`	Student dashboard: enrolled courses, continue-learning card, pending orders, upcoming live sessions.
`/checkout/[orderId]`	Bank transfer instructions or CMI launch view.

`/checkout/cmi/[orderId]`	Auto-submits the signed CMI form.
`/checkout/[orderId]/confirmation`	Post-payment landing page.

5.2 Admin routes (ADMIN only)

Path	Purpose
`/admin`	Dashboard overview.
`/admin/site`	Edit every public-site string and asset. Tabbed: Brand, Hero, Trust strip, Urgency banner, Mid-CTA, Final CTA, Footer (incl. Contact), SEO, Bank transfer, Stripe, CMI.
`/admin/courses`	Course CRUD with filterable table.
`/admin/courses/[id]`	Module/lesson editor, pricing, status, payouts.
`/admin/live`	Live session CRUD.
`/admin/consultants`	Consultant profiles + availability editor.
`/admin/users`	User table with bulk delete, role change, force sign-out.
`/admin/orders`	Order ledger.
`/admin/orders/[id]`	Order detail: confirm payment, cancel, refund, raw CMI callback inspection.
`/admin/grading`	Pending quiz/assignment submissions awaiting manual grading.
`/admin/payouts`	Instructor payout ledger with mark-paid action.
`/admin/analytics`	Revenue + enrollment metrics with CSV export.
`/admin/pages`	CMS page editor (privacy, terms, etc.).

5.3 Instructor routes (INSTRUCTOR + ADMIN)

Path	Purpose
`/instructor`	Read-only dashboard: students, earnings, pending grading queue, recent enrollments.

5.4 API routes

Path	Purpose
`POST /api/webhooks/stripe`	Stripe webhook. Verifies signature, fulfills paid orders, handles refunds.
`POST /api/cmi/callback`	CMI hosted-page callback. Verifies SHA-512 hash, fulfills paid orders.
`GET /api/cron/live-session-reminders`	Cron endpoint. Requires `Authorization: Bearer ${CRON_SECRET}`. Sends T-60min reminders.
`POST /api/uploadthing`	UploadThing direct upload handler.
`/api/auth/[...nextauth]`	NextAuth handlers.
`GET /api/admin/analytics/export`	CSV export for the analytics dashboard.

6. Authentication & Authorization

6.1 Identity providers

- **Email magic-link** (default): user enters email → magic link delivered via Resend → click to sign in. Tokens valid for 24h.
- **Google OAuth** — standard OAuth flow via NextAuth.
- **LinkedIn OAuth** — standard OAuth flow via NextAuth.

All providers map to a single `User` record by email.

6.2 Role-based access control

Three roles, hierarchical:

- `STUDENT` — default. Can enroll, watch, review, book live + consults.
- `INSTRUCTOR` — can access `/instructor` (read-only). Owns courses + live sessions. Admins do the editing.
- `ADMIN` — full access to `/admin/*`. Configured by setting `User.role` in DB.

Layouts enforce the gate:

- `app/(admin)/admin/layout.tsx` — redirects non-admins to `/dashboard`.
- `app/instructor/layout.tsx` — redirects STUDENTs to `/dashboard`.
- `app/(learn)/courses/[slug]/learn/page.tsx` — `requireEnrollment(slug, courseId)` redirects unenrolled to the marketing page.

6.3 Server actions

Every mutation is a server action (`"use server"`). All actions re-check authorization before mutating; client-side hiding of buttons is for UX only, never security.

7. Security

7.1 Transport

- HTTPS-only (Vercel-managed Let's Encrypt).
- `Strict-Transport-Security: max-age=86400; includeSubDomains` header.
- `308` permanent redirect from apex (`jissron.com`) to `www.jissron.com` .

7.2 Content Security Policy

Restrictive CSP set in `next.config.ts` :

- `default-src 'self'`
- `script-src` allows inline + eval (Next.js requirement) + Bunny + UploadThing + unpkg.
- `img-src` allows known CDNs (Bunny, UploadThing, Google OAuth avatars).
- `frame-src` restricted to Bunny video frames and UploadThing iframes.
- `form-action 'self'` , `frame-ancestors 'self'` , `object-src 'none'` .

7.3 Payment signature verification

- **Stripe**: every webhook event is verified against `STRIPE_WEBHOOK_SECRET` using `stripe.webhooks.constructEvent` . Unverified events return 401.
- **CMI**: every callback body has its `HASH` field re-computed from the alphabetically-sorted form params + `cmiStoreKey` , joined with `|` (with escaping for `|` and `\`), SHA-512 base64. Mismatched hashes redirect to error. Comparison uses `crypto.timingSafeEqual` .

7.4 PCI scope

We use hosted payment pages exclusively (Stripe Checkout + CMI's hosted page). **No card data ever touches our servers.** This puts us in **PCI-DSS SAQ A** scope — the lightest tier.

7.5 Secrets

- All secrets live in Vercel Environment Variables, never in code.
- `.env.local` is git-ignored.
- Stripe webhook secret + CMI store key + Resend API key + UploadThing tokens + Supabase service-role key.

7.6 Cron endpoint protection

`/api/cron/live-session-reminders` requires `Authorization: Bearer ${CRON_SECRET}`. Without the header, the endpoint returns 401. Without the env var set on the server, it returns 503. Header comparison is constant-time-safe (string equality on a known prefix).

7.7 Anti-abuse

- Contact form has a hidden honeypot field (`website`) that bots fill; humans don't see it. Filled submissions silently succeed without sending.
- Bulk-delete on `/admin/users` filters out the admin's own user ID before processing.
- Live session bookings use `Serializable` transaction isolation when counting seats, so concurrent purchases cannot oversell.

7.8 Data exfiltration controls

- Bunny.net video API key is server-only — embed URLs are signed server-side before being passed to the browser.
- Stripe secret key is server-only — only the publishable key reaches the client (and only when explicitly needed for client-side checkout flows, which we don't currently use).
- UploadThing routes enforce per-endpoint size and type limits.

8. Payments

8.1 Order lifecycle

All paid transactions flow through one `Order` model:

```
PENDING → PAID → (REFUNDED)
        ↘ CANCELLED
        ↘ EXPIRED (after timeout window)
```

Expiry windows per payment method:

- Bank transfer: 7 days
- CMI: 1 hour
- Stripe: 24 hours

A nightly read of `findAndMarkExpiredOrders` (triggered on dashboard / admin views) flips overdue PENDING orders to EXPIRED and sends the customer an email.

8.2 Fulfillment

A single function `fulfillPaidOrder({ orderId, provider })` in `lib/actions/fulfill-order.ts` is the authoritative post-payment provisioning step. It is called from:

- Stripe webhook (`checkout.session.completed`)
- CMI callback (`isCmiCallbackApproved`)
- Admin bank-transfer confirm action

It dispatches on which FK is set on the Order:

- `courseId` → upserts `Enrollment(status: ACTIVE)` .
- `liveSessionId` → re-checks seat capacity inside the transaction, then upserts `Booking(status: CONFIRMED)` .
- `consultBookingId` → flips the `ConsultBooking` to `CONFIRMED` .

Idempotent: if the order is already PAID, the function returns immediately.

8.3 Refunds

Admin-initiated via `/admin/orders/[id]` :

- For **Stripe** orders: looks up the linked `PaymentIntent` via metadata, calls `stripe.refunds.create`, marks the order REFUNDED.
- For **CMI** and **bank transfer**: only flips the DB status. Admin is responsible for the actual money-out via their CMI dashboard or bank.

Side effects in all cases:

- `Enrollment` is moved to `REVOKED` (student loses access).
- `Booking` is deleted (frees the live-session seat).

9. File Hosting

9.1 UploadThing endpoints

Defined in `app/api/uploadthing/core.ts`. Each endpoint sets type and size constraints:

- `userAvatar` — 2 MB, image only.
- `courseThumbnail` — 4 MB, image.
- `lessonAudio` — 64 MB, audio.
- `lessonPdf` — 32 MB, application/pdf.
- `assignmentSubmission` — 32 MB, mixed types.

9.2 Video hosting

Course videos live on **Bunny.net Stream**. Each lesson stores a `videoGuid`. At render time, `lib/bunny.ts` generates a signed embed URL valid for the session. The API key never reaches the browser.

10. Email

All transactional email goes through **Resend**. Templates live in `lib/emails/senders.ts`:

- `sendOrderReceived` — bank transfer instructions after PENDING order created.
- `sendPaymentConfirmed` — receipt on order → PAID transition.
- `sendOrderExpired` — sent when a stale PENDING order is auto-expired.
- `sendCourseCompleted` — sent when a student completes 100% of lessons. Includes deep link to review modal.
- `sendLiveSessionReminder` — T-60min reminder before a live session. Fired by external cron with idempotency stamp.

Sender address configurable via `EMAIL_FROM` env var. Production should use a verified `jissron.com` sender.

11. Background Jobs

11.1 Live session reminder cron

Hobby-tier Vercel doesn't permit sub-daily cron schedules. Instead, an external service (**cron-job.org**) is configured to hit `/api/cron/live-session-reminders` every 5 minutes with the `Authorization: Bearer ${CRON_SECRET}` header.

The endpoint:

- Queries Bookings whose linked `LiveSession.startsAt` is within the next 60 minutes AND `reminderSentAt IS NULL` AND status is SCHEDULED/LIVE.
- Sends the reminder email via Resend.
- Stamps `reminderSentAt` for idempotency.

Returns `{ ok, scanned, sent, failed, horizon }` for observability.

11.2 Order auto-expiry

Triggered on read by `autoExpireOrders()` calls in the dashboard and admin-orders page. Cheap, no cron required.

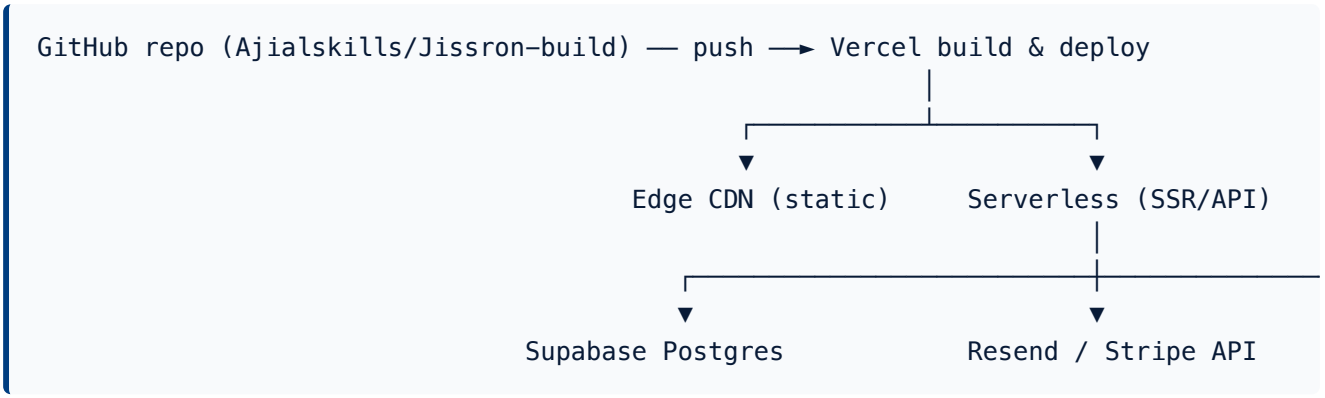
12. Environment Variables

Variable	Required	Purpose
`DATABASE_URL`	✓	Supabase pooler URL (runtime queries).
`DIRECT_URL`	✓	Supabase direct URL (Prisma migrations).
`NEXTAUTH_URL`	✓	Canonical site URL, e.g. `https://www.jissron.com`.
`NEXTAUTH_SECRET`	✓	Random 32-byte secret for JWT signing.
`RESEND_API_KEY`	✓	Resend transactional email.
`EMAIL_FROM`	✓	Verified sender, e.g. `JissrON <hello@jissron.com>`.
`STRIPE_SECRET_KEY`	optional	Stripe server-side. If absent, DB `SiteSettings.stripeSecretKey` is read as fallback.
`STRIPE_WEBHOOK_SECRET`	optional	Verifies incoming Stripe webhook signatures.
`UPLOADTHING_TOKEN`	✓	UploadThing API.
`BUNNY_STREAM_LIBRARY_ID` + `BUNNY_STREAM_API_KEY`	optional	Required only if videos are hosted on Bunny.
`GOOGLE_CLIENT_ID` + `GOOGLE_CLIENT_SECRET`	optional	Google OAuth.
`LINKEDIN_CLIENT_ID` + `LINKEDIN_CLIENT_SECRET`	optional	LinkedIn OAuth.
`CRON_SECRET`	✓ for reminders	Bearer token for the reminder cron endpoint.

CMI credentials (`cmiMerchantId` , `cmiStoreKey`) live in the DB (`SiteSettings`), not env, since they're managed via `/admin/site` .

13. Deployment

13.1 Topology



13.2 Promotion

- Default branch: `main`.
- Auto-deploy: enabled (via Vercel GitHub App).
- Database migrations: run manually via `pnpm prisma db push` against `DIRECT_URL` before deploying schema changes.

13.3 Observability

- Vercel function logs for runtime errors.
- `app/(marketing)/error.tsx` catches public-site render errors and shows a graceful page with the Next.js error digest.
- `ActivityLog` table for admin actions.

14. Conventions

- **Currency**: Always stored as integer cents (`priceMadCents` , `priceUsdCents`). Display formatting via `lib/currency.ts` .
- **Identifiers**: cuid (`@default(cuid())`) for primary keys. Order references are human-readable counters from the `Counter` model.
- **Timestamps**: All UTC, stored as `DateTime` . Displayed in user's locale via `date-fns` .
- **Booleans**: Affirmative phrasing (`acceptsNew` , `isFree` , `isFeatured`) rather than negative.
- **Server actions**: All in `lib/actions/*.ts` . All marked `"use server"` at the top of file. Always return `{ ok: true, ... } | { ok: false, error }` .

15. Known Limitations

- **Hobby plan cron**: Sub-daily Vercel crons are blocked. Workaround: cron-job.org (see §11).
- **Supabase RLS**: Currently disabled across all tables. Safe with Prisma-only access (server-side service key), but must be enabled before adding any browser-side Supabase client.
- **Reviews require completion**: Students must finish 100% of lessons before reviewing. By design, but constrains review volume.
- **Slot timezone**: Consultant availability is stored and displayed in UTC. A future enhancement could detect viewer timezone and translate.
- **No multi-instructor**: A course has exactly one instructor. Co-teaching would require a join table.